

Processor Systems Design – Assignment 1

Amit Barman, 27002
amitbarman@iisc.ac.in

10 September 2026

Laboratory Exercise 1

Design and implement a 16-bit Multicycle MC16 Processor on Digilent BASYS3 FPGA Board. Target Device is Xilinx Artix-7 XC7A35T- ICPG236C (Family Artix-7, Part XC7A35T, Package CPG236, Speed Grade -1). Refer to the multi-cycle CISC processor case study discussed in the course.

The Processor is a 16-bit CISC CPU with 5-bit address space with three operand register instructions. The instructions are 16-bit and Datapath is 16-bits wide. The processor is a multi-cycle processor which does fetch, decode, execution, and write steps sequentially in multiple cycles. It has eight registers and flag zero and carry flags. Flags need not be a readable register. Conditional jump instructions can use the flags. Design it based on the multi-cycle processor discussed in the class.

It should support minimal arithmetic-logic, load, store and jump/branch instructions with a program counter. (load, store, add, sub, increment, decrement, shift-left, jump-on-zero, jump on-non-zero, jump-on-carry, jump-on-no-carry, etc.) It must support jump (branch) on condition to implement loops. Assume three operand instructions. It need not have external interrupts or exception handling or function calling (i.e. no stack support). The processor has an Instruction prefetch register (IPR) available, the output of which goes to instruction register (IR) so that instruction fetch of the next instruction can happen concurrently with the processing of the current instruction.

The architecture should be similar to the one discussed in class and should have only a single shared data bus for all data movement and should not have dedicated connection between units. Since the FPGA does not have internal tri-state gates, use a regular multiplexer to multiplex outputs on to the data bus. Instruction-cum-data memory can be 64 x 16 bit. Implement it using Distributed RAM with asynchronous read. You can use 5-bit opcode and three bits each to specify destination, source-1 and source-2 registers. Load, store, jump, jump-on-zero, and jump-on-non-zero can use 5-bit addresses. You are required to implement the instructions needed for the following test program. Feel free to modify the instruction set.

Test the processor with a program to do the unsigned division of two 16-bit numbers. The program can be terminated with a jump instruction jumping to the address of the same instruction. Use “coe” file to load instruction binary and data in the memory.

Design the Datapath block schematic (Level 0 and Level 1) and FSM state diagram first. Then do the Verilog coding. Give the IO Constraints and Timing Constraints to achieve maximum delay performance. Do the Timing Analysis and Timing simulation with proper test bench. Show the program working on BASYS3 FPGA Board. Provide reset on a push button. Display the result on discrete LEDs or seven segment display.

Solution :

0.1 Division Algorithm and Assembly Code

Considering two 16-bit unsigned numbers $A = a_{15}a_{14}.....a_2a_1a_0$ and $B = b_{15}b_{14}.....b_2b_1b_0$, Assuming the processor performs unsigned integer division,

$$A \div B = Q(\text{ with remainder } R)$$

'A' can be expressed as,

$$A = B.Q + R$$

Where, $0 \leq R < B$, $A \implies$ Dividend, $B \implies$ Divisor, $Q \implies$ Quotient and $R \implies$ Remainder.

For mathematical basis, n -bits operation

$$A = \sum_{i=0}^{n-2} a_i \cdot 2^i \qquad B = \sum_{i=0}^{n-2} b_i \cdot 2^i$$

where $a_i, b_i \in \{0, 1\}$ Similarly the Quotient will be -

$$Q = \sum_{i=0}^{n-2} q_i \cdot 2^i$$

At each step, a partial remainder R_i is formed:

$$R_{i+1} = R_i - B$$

0.2 Pseudocode for Division Operation

Input: A (16-bit dividend), B (16-bit divisor)

$R \leftarrow A$
 $Q \leftarrow 0$

LOOP:
 $T \leftarrow R - B$
If $T \geq 0$:
 $R \leftarrow T$
 $Q \leftarrow Q + 1$
Go to LOOP

Else:
Stop

Output: $Q = \text{Quotient}$, $R = \text{Remainder}$

0.3 The Assembly code for this 16-bit Multicycle MC16 Processor

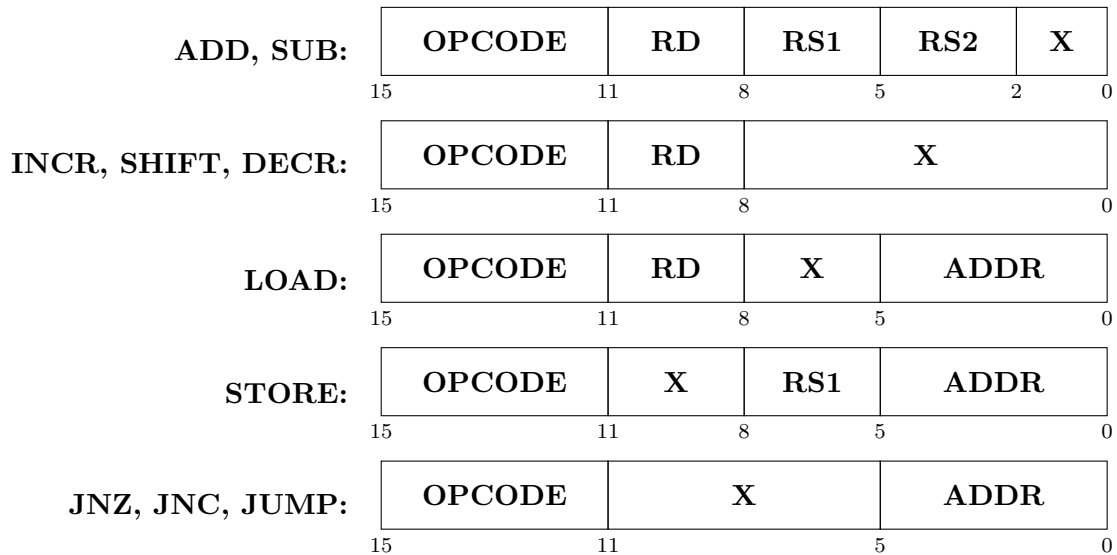
In the Assembly code, there are two different types of LOAD operations. One is LOAD R1, 2FH $\rightarrow$ which denotes that the value of 2F (in Hexadecimal) will be loaded to R1 register, and the 2F value will be stored immediately next to the location of the mnemonic LOAD R1, 2FH. Meanwhile by using LOAD R4, 0x1EH $\rightarrow$ it denotes that the value which is in the location 0x1E of DRAM will be stored to R4 register. In One word,

2FH $\rightarrow$ The value in hexadecimal & $0 \times 1EH \rightarrow$ The Address of DRAM

1	ORG 0	
2	LOAD R1, 2FH	; R1 <= dividend (= 47)
3	LOAD R2, 05H	; R2 <= divisor (= 5)
4	SUB R3, R2, R2	; R3 <= 0 (initial quotient)
5	LOOP: SUB R1, R1, R2	; tentative subtraction (R1 <= R1 - R2)
6	JC 0x10H	; carry=1: valid subtraction
7	ADD R1, R1, R2	; undo subtraction if borrow
8	JUMP DONE	; division complete
9	0x10H: INCR R3	; quotient = quotient + 1
10	JUMP LOOP	; repeat subtraction
11	DONE: STORE R1, 0x1FH	; store remainder
12	STORE R3, 0x1EH	; store quotient
13	LOAD R4, 0x1EH	; R4 <= quotient
14	LOAD R5, 0x1FH	; R5 <= remainder
15	HALT	; End

0.4 Instruction Architecture

Instructions are 16 bits long, with a 5-bit opcode, 3 bits each for the destination and source registers, and a 5-bit address for branch, load and store instructions. The Instruction Architecture is shown below.



0.5 Datapath Block Diagram

The datapath contains a set of interconnected components that handle data processing and transfers. These components include eight registers, two temporary registers, an Arithmetic and Logic Unit (ALU), Carry and Zero Flags, a Program Counter, a Memory Address Register, an Instruction Register and Decoder, and the Instruction-cum-Data Memory. All these blocks connect to a common databus, with a multiplexer controlling access. While the datapath performs all data manipulation and transfers, the Controller manages the timing and sequencing of these operations. The block diagram is shown in Figure 2.

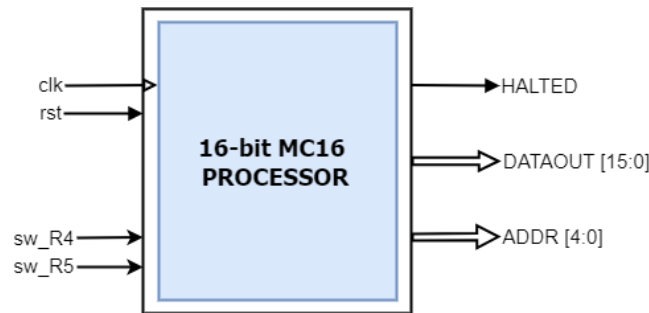


Figure 1: Block diagram (Level 0) of the 16-bit MC16 Processor

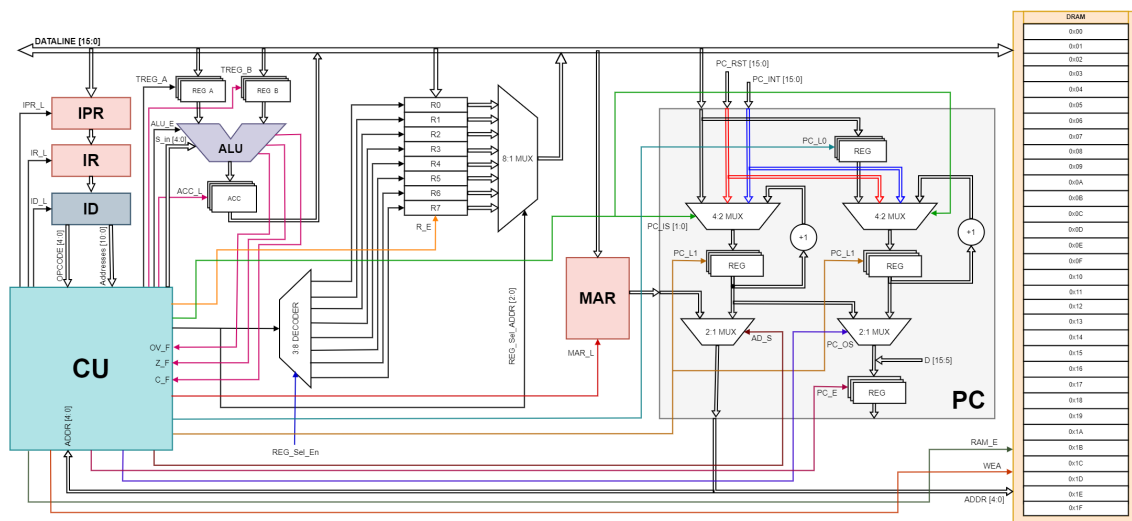


Figure 2: Block Diagram (Level 1 & 2) of the Datapath of this 16-bit Multicycle MC16 Processor

0.6 Control Unit Finite State Machine

The timing signals are controlled by the Controller of the Processor. It only generates control signals based on the instruction using a Finite State Machine; it does not take part in any computation. The Controller FSM State Transitions for this processor are shown in Figure 3, and the outputs corresponding to each state are shown in Table 1.

0.7 OPCODE and Their Operations

The entire list of different types of opcodes is listed here in Table 2 -

0.8 Timing Report Summary

The clock period for the design is set to be $14ns$, i.e. the frequency of operation is $71.428MHz$. In this condition, the timing report summary is as shown in Figure 4. Here we can see that there's a positive slack of $5.848ns$, which implies that the maximum theoretical clock frequency for this design is,

$$f_{max} = \frac{1}{(14 - 5.848) \times 10^{-9}} \cdot s = 122.67 MHz$$

Table 1: State Outputs table for Controller FSM

State	Description	Output Signals
S_IDLE	IDLE state of Processor	All signals are zero; PC.IS = PC.RESET (10)
S_WAIT1	Waiting state to get data from DRAM (1 clk latency)	RAM.E = 1; AD.S = 1
S_PREFETCH	Instruction Prefetch Ops	IPR.L = 1; RAM.E = 1; AD.S = 1; PC.L1 = 0; MAR.L = 1
S_FETCH	Instruction Fetch Ops	IPR.L = 1; IRL = 1; RAM.E = 1; AD.S = 1; PC.L1 = 1
S_IDLOAD	Loading instruction in Instruction Decoder(ID)	ID.L = 1; PC.L1 = 0; RAM.E = 1
S_DECODE	Instruction decoding Ops	All outputs are in defaults state
S_LOAD_VAL	Starts the immediate data loading	RAM.E = 1; PC.L1 = 0
S_LOAD_WRITEBACK	Loading of data into destination register	Reg_Sel_addr = RD; Reg_Sel_En = 1; REG.L = 1; PC.L1 = 1; AD.S = 1
S_LOAD_DIRECT	Address for direct load Ops	AD.S = 0
S_LOAD_DIRECT_RD	Start getting the data	AD.S = 0; RAM.E = 1
S_LOAD_DIRECT_WB	Loading of data directly at destination register	AD.S = 0; RAM.E = 1; Reg_Sel_addr = RD; Reg_Sel_En = 1; REG.L = 1;
S_MEMWRITE_SETUP	Store Ops initialisation	WEA = 0; RAM.E = 1; AD.S = 0; Reg_Sel_addr = RS1; Reg_Sel_En = 1; REG.L = 0;
S_MEMWRITE	Store in DRAM Ops	RAM.E = 1; WEA = 1; AD.S = 0; Reg_Sel_addr = RS1; Reg_Sel_En = 1; REG.L = 0;
S_LOAD_TREGA	Load data at REG A	Reg_Sel_addr = RD or RS1; Reg_Sel_En = 1; REG.L = 0; TREG.A = 1; AD.S = 1
S_LOAD_TREGB	Load data at REG B	Reg_Sel_addr = RS2; Reg_Sel_En = 1; REG.L = 0; TREG.A = 1
S_COMPUTE_1	ALU execution start	ALU.E = 1
S_COMPUTE_2	Load execution results to Accumulator	ALU.E = 1
S_ALU_WRITEBACK	Load ALU result to Register	ACC.L = 1; Reg_Sel_addr = RD; Reg_Sel_En = 1; REG.L = 1
S_JCOND	Branching Ops (JUMP, JZ, JNZ, JC, JNC)	AD.S = 1; PC.IS = PC.LOAD (11); PC.L1 = 1
S_HALT	Halt Ops or End of Execution	HALTED = 1

0.9 Resource Utilization Report

The summary of resource utilization by the design on the Basys3 FPGA Board is shown in Figure 5. The detailed hierarchy is shown in Figure 6. From Figure 6, we can see that the datapath occupies the maximum number of LUTs and flip-flops, and the controller consumes less area than the datapath. The 5 registers in the controller correspond to 5 states, since

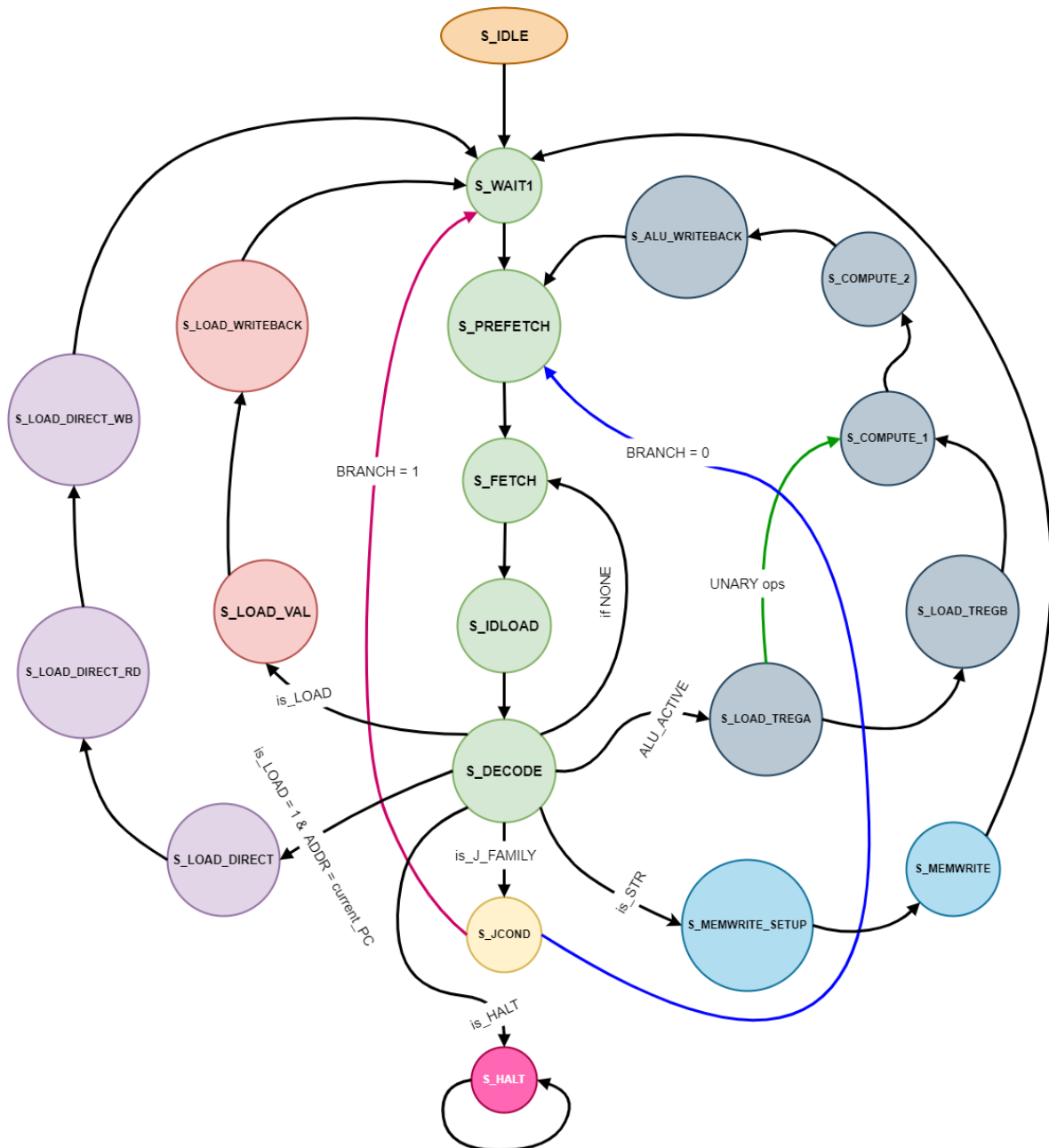


Figure 3: Controller FSM State Transition Diagram of this 16-bit Multicycle MC16 Processor

Vivado optimises the FSM to one-hot encoding to simplify hardware implementation. Also, the distributed RAM corresponds to LUTRAM, and for 32×16 – *bit* memory it consumes 16 LUTRAMs. One more observation is that in the hierarchy the instruction decoder module is not shown, which results from the fact that it is nothing but rewiring the instruction signals.

0.10 Post-Implementation Timing Simulation

Post-implementation timing simulation was performed on the designed 16-bit multicycle processor to verify its functionality under realistic timing conditions. Unlike functional simulation,

Table 2: OPCODE Table and their details

OPCODE	Binary forms	Operation	E.g. Mnemonics
INCR	00001	Increment a value in a register by +1	INCR R1
ADD	00010	Addition of two values and store in a register	ADD R0, R1, R6
ADC	00011	Addition of two values with carry and store in a register	ADC R1, R5, R3
SUBC	00100	Subtraction of two values with borrow and store in a register	SUBC R0, R7, R4
SUB	00101	Subtraction of two values and store in a register	SUB R5, R4, R1
DECR	00110	Decrement a value in a register by +1	DECR R2
AND	01000	AND operation between two values and store in a register	AND R0, R1, R6
OR	01010	OR operation between two values and store in a register	OR R2, R1, R3
XOR	01100	XOR operation between two values and store in a register	XOR R7, R1, R6
NOT	01110	NOT operation of a value and store in that register	NOT R5
SHL	10000	Logical left shift	SHL R5
SHR	10010	Logical right shift	SHR R3
ASHL	10100	Arithmetic left shift	ASHL R4
ASHR	10110	Arithmetic right shift	ASHR R7
JUMP	11000	Branching to another location or label	JUMP 0x1CH ; JUMP NEXT
LOAD	11001	(1) Loading of data from the immediate next location (2 words) (2) Loading of data from a defined address (1 word)	LOAD R4, 45H LOAD R6, 0x13H
STORE	11010	Stores data from a register to DRAM	STORE R1, 0x1DH
JZ	11011	Jump to location if Zero (zero flag = 1)	JZ 0x12H ; JZ LOOP
JNZ	11100	Jump to location if Not Zero (zero flag = 0)	JNZ 0x0FH ; JNZ LOOP
JC	11101	Jump to location if Carry is present (carry flag = 1)	JC 0x07H ; JC LATER
JNC	11110	Jump to location if No Carry (carry flag = 0)	JNC 0x09H ; JNC DONE
HALT	11111	End of Instructions/Execution	HALT

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.848 ns	Worst Hold Slack (WHS): 0.088 ns	Worst Pulse Width Slack (WPWS): 5.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 748	Total Number of Endpoints: 748	Total Number of Endpoints: 354

All user specified timing constraints are met.

Figure 4: Timing Summary of the 16-bit Multicycle MC16 Processor

Summary

Resource	Utilization	Available	Utilization %
LUT	285	20800	1.37
LUTRAM	16	9600	0.17
FF	371	41600	0.89
IO	26	106	24.53

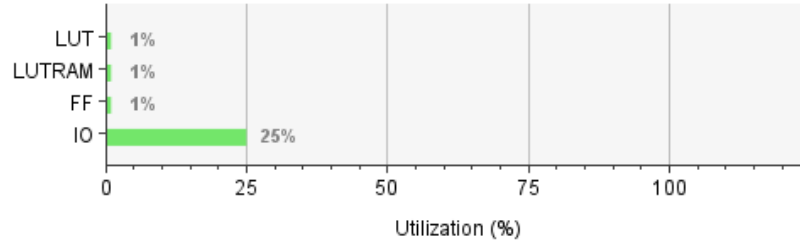


Figure 5: Resource utilization summary of the MC16 16-bit Processor

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	Bonded IOB (106)	BUFGCTRL (32)
MC16_Processor	285	371	22	116	269	16	26	2
ctrl_unit (MC16_control_unit)	67	5	0	38	67	0	0	0
datapath_block (dataPATH)	218	350	22	113	202	16	0	0
alu_unit (ALU16)	132	89	6	47	132	0	0	0
reglistR (RegLists)	48	128	16	49	48	0	0	0

Figure 6: Hierarchy of resource utilization of MC16 16-bit Processor

which mainly verifies the logical behaviour of the processor, post-implementation simulation includes the timing characteristics of the synthesized and implemented design, such as propagation delays and clock-to-output delays. The simulation verified the operation of the major processor components: the Program Counter (PC), Instruction Register (IR), Instruction Pre-Fetch Register (IPR), Memory Address Register (MAR), register file, Arithmetic Logic Unit (ALU), control unit (CU), decoder, and multiplexers. The simulated output waveforms are given below.

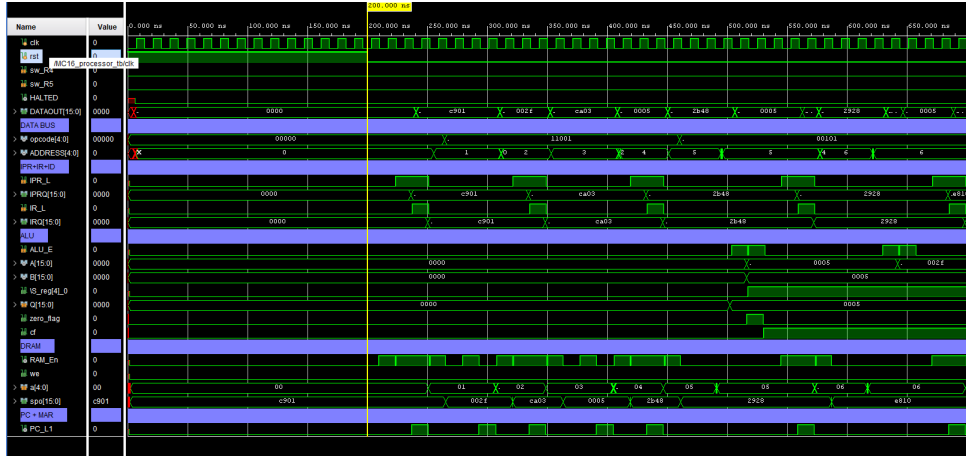


Figure 7: Post-implementation waveforms (1) during Simulation start

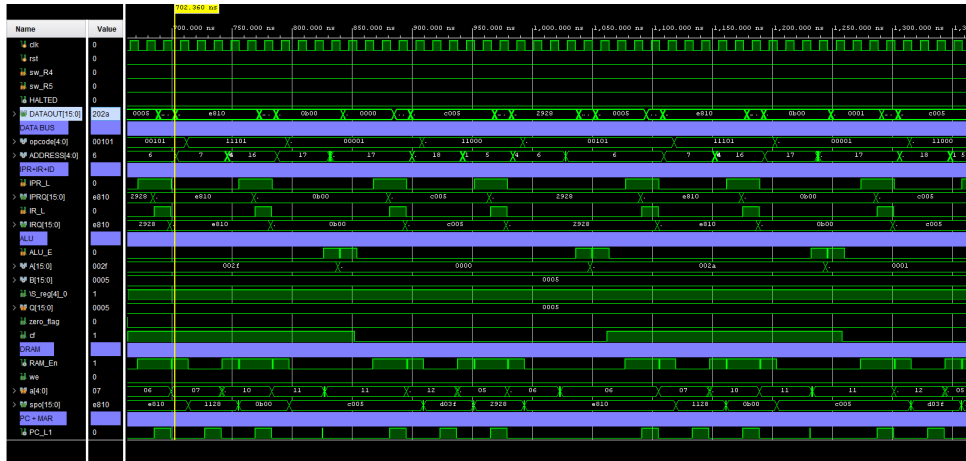


Figure 8: Post-implementation waveforms (2) during loop operation

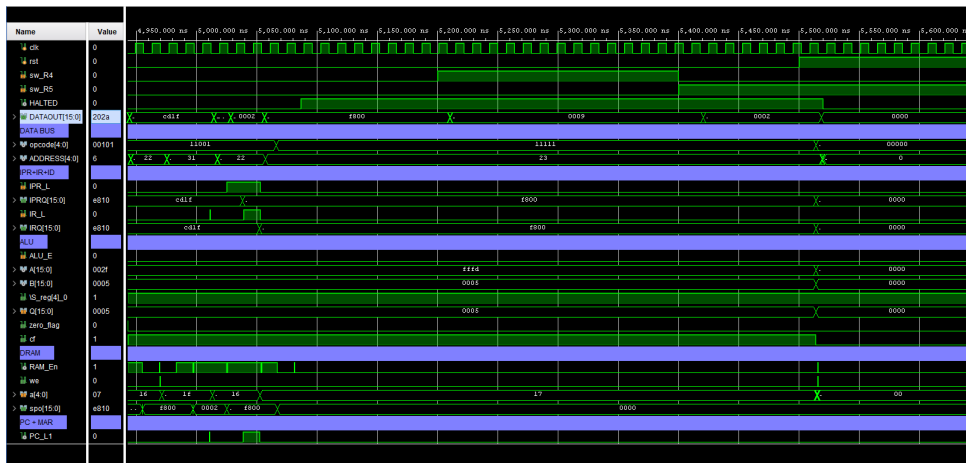


Figure 9: Post-implementation waveform (3) output results